C:\hpc-23-feb-2026\.\28\Server.java

```
// This is a very bad example of server programming
import java.io.*;
import java.net.*;
class Server
{
public static void main(String gg[]) throws Exception
{
ServerSocket serverSocket=new ServerSocket(3500);
Socket clientSocket;
OutputStream outputStream;
OutputStreamWriter outputStreamWriter;
while(true)
{
System.out.println("Server is waiting to accept request...");
clientSocket=serverSocket.accept();
System.out.println("Server accepted a connection");
outputStream=clientSocket.getOutputStream();
outputStreamWriter=new OutputStreamWriter(outputStream);
outputStreamWriter.write("Hello, Welcome to the SERVER");
outputStreamWriter.flush();
outputStream.close();
clientSocket.close();
}
}
}
// compile using javac Server.java
// run using java Server
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\scaler\create_random_csv.c

```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
#define FILES_COUNT 5000
int main()
{
int x,rows,columns,r,c;
char file_name[51];
FILE *file;
int min,max;
int value;
srand(time(NULL));
for(x=1;x<=FILES_COUNT;++x)
{
sprintf(file_name,"source/data_%d.csv",x);
file=fopen(file_name,"wb");
min=3; max=45;
columns=rand()%(max-min)+min;
min=1000; max=9000;
rows=rand()%(max-min)+min;
min=1;
max=1000000;
for(r=1;r<=rows;++r)
{
for(c=1;c<columns;++c)
{
value=rand()%(max-min)+min;
fprintf(file,"%d,",value);
}
value=rand()%(max-min)+min;
fprintf(file,"%d\n",value);
}
fclose(file);
printf("File %s created with %d rows and %d columns\n",file_name,rows,columns);
}
return 0;
}
```

C:\hpc-23-feb-2026\.\scaler\list_of_csvs.c

```
#include<stdio.h>
#include<dirent.h>
#include<string.h>
int main()
{
DIR *directory;
struct dirent *directory_entry;
char *ptr;
directory=opendir("source");
if(directory==NULL)
{
printf("Unable to access source directory\n");
return 1;
}
while(1)
{
directory_entry=readdir(directory);
if(directory_entry==NULL) break;
ptr=strrchr(directory_entry->d_name,'.');
if(ptr && strcmp(ptr,".csv")==0)
{
printf("%s\n",directory_entry->d_name);
}
}
closedir(directory);
return 0;
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\scaler\read_contents.c

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
//#define BUFFER_SIZE 1024*1024
#define BUFFER_SIZE 1024
int main()
{
char file_name[51];
FILE *file;
unsigned char *buffer;
size_t read_count;
int i;
buffer=(unsigned char *)malloc(BUFFER_SIZE);
if(buffer==NULL)
{
printf("Low memory\n");
return 1;
}
strcpy(file_name,"source/data_1.csv");
file=fopen(file_name,"rb");
if(file==NULL)
{
printf("Unable to access %s\n",file_name);
return 1;
}
while(1)
{
read_count=fread(buffer,1,BUFFER_SIZE,file);
if(read_count==0) break;
printf("Bytes extracted %lu\n",read_count);
//for(i=0;i<read_count;++i) printf("%c",buffer[i]);
}
fclose(file);
free(buffer);
return 0;
}
```

C:\hpc-23-feb-2026\.\scaler\scale_one.c

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
//#define BUFFER_SIZE 1024*1024
#define BUFFER_SIZE 1024
int main()
{
char file_name[51];
FILE *file;
unsigned char *buffer;
unsigned char c;
size_t read_count;
int number_of_columns;
double *min,*max;
int i;
buffer=(unsigned char *)malloc(BUFFER_SIZE);
if(buffer==NULL)
{
printf("Low memory\n");
return 1;
}
strcpy(file_name,"source/data_1.csv");
file=fopen(file_name,"rb");
if(file==NULL)
{
printf("Unable to access %s\n",file_name);
return 1;
}
number_of_columns=1;
// 100% file will contain more than 1 row separated by \n
do
{
read_count=fread(buffer,1,BUFFER_SIZE,file);
for(i=0;i<read_count;++i)
{
c=buffer[i];
if(c=='\r') continue;
if(c=='\n') break;
if(c==',') ++number_of_columns;
}
}while(c!='\n')
rewind(file); // internal pointer will go to starting position (0)
min=(double *)malloc(sizeof(double)*(number_of_columns-1)); // last column not to be scaled
if(!min)
{
printf("Low memory\n");
exit(1);
}
```

```
max=(double *)malloc(sizeof(double)*(number_of_columns-1)); // last column not to be scaled
if(!max)
{
free(min);
printf("Low memory\n");
exit(1);
}
//code to read first row and and assign all doubles (except last one) to min and max
// code to read the rest of the rows and find min and max
// rewind(file)
// open target file in wb mode
// start reading and apply scaler on each value and write to target file
// close files
fclose(file);
free(buffer);
return 0;
}
```

C:\hpc-23-feb-2026\.\28\client.c

```
#include<stdio.h>
#include<sys/socket.h>
#include<string.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<unistd.h>
int main()
{
int file_descriptor;
struct sockaddr_in server_address;
char buffer[1025];
int n;
file_descriptor=socket(AF_INET,SOCK_STREAM,0);
if(file_descriptor<0)
{
printf("Unable to create socket\n");
return 1;
}
// initialize all the bytes of server_address structure to zero
bzero(&server_address,sizeof(server_address));
server_address.sin_family=AF_INET;
server_address.sin_port=htons(3500);  // server port
if(inet_pton(AF_INET,"192.168.56.1",&server_address.sin_addr)<0)
{
printf("Unable to setup server ip\n");
return 1;
}
if(connect(file_descriptor,(struct sockaddr *)&server_address,sizeof(server_address))<0)
{
printf("Unable to connect to server\n");
return 1;
}
n=read(file_descriptor,buffer,1024);
if(n==0)
{
printf("Server is not saying anything\n");
}
else
{
buffer[n]='\0';
printf("Server said [%s]\n",buffer);
}
return 0;
}
```